

Capturing Complex Patterns in Nepali NLP via Stable Polynomial FFN

Gunjan Thapa

gunjanthapa5308a@gmail.com

Siddhant Bhattarai

sidftww@gmail.com

Abstract—This paper investigates the efficacy of 2nd-degree parametric polynomial activation functions within the Feed-Forward Network (FFN) layers of Transformer architectures for Nepali Natural Language Processing. While standard linear or piecewise activations like ReLU and GeLU dominate contemporary language modeling, they scale poorly when modeling high-order morphological structures inherent in the Devanagari script. We propose a parameterized decoder architecture utilizing multi-degree feature maps stabilized by a dual global norm-based gradient clipping mechanism and an exponential gradient control framework.

1 INTRODUCTION

Nepali is a low-resource language characterized by complex inflectional systems and extensive morphological modifications, particularly apparent at the verbal root (*Dhatu*) level. Conventional Large Language Models (LLMs) rely heavily on piecewise linear transformations to route information through internal Feed-Forward networks. However, these static mappings scale linearly with depth, often lacking the parameter density to articulate high-order semantic features without dramatically compounding overall parameter budgets.

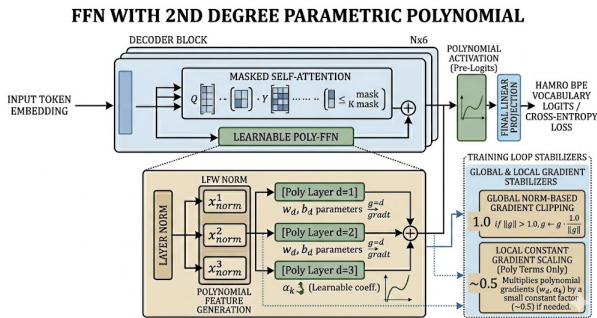


Figure 1: Loss over iteration during training.

To overcome this structural barrier, this work introduces a learnable polynomial architecture designed to dynamically expand feature embeddings into localized higher-order dimensional vectors. By exchanging linear layers for multi-lane parametric polynomials, the transformer blocks map nuanced linguistic paradigms more natively.

2 POLYNOMIAL FORWARD PASS

The structural core of the proposed transformer decoder replaces standard multi-layer perceptron blocks with a parallelized *Learnable Parametric Polynomial Feed-Forward Network* (Learnable Poly-FFN).

Let $x \in \mathbb{R}^{d_{\text{model}}}$ denote the hidden vector state processed through the initial layer normalization boundary. As outlined in the architecture topology, the normalized representation is explicitly split into three distinct orders of a polynomial expansion, defining a multi-lane feature set $\{x_{\text{norm}}^1, x_{\text{norm}}^2, x_{\text{norm}}^3\}$. Each individual component lane scales independently according to the following generalized mapping equation:

$$f_d(x_{\text{norm}}^d) = w_d \cdot x_{\text{norm}}^d + b_d \quad \forall d \in \{1, 2, 3\} \quad (1)$$

where w_d and b_d are degree-specific weights and biases dedicated to isolating structural shifts across different dimensions of the feature tensor.

Concurrently, a non-linear modulation trajectory processes the input states via an arbitrary learned scalar vector coefficient α_k . The comprehensive forward activation path across the combined parametric terms is computed algebraically through element-wise summation:

$$f_{\text{poly}}(x) = \sum_{d=1}^3 (w_d \cdot x_{\text{norm}}^d + b_d) + \psi(\alpha_k) \quad (2)$$

The aggregated output tensor is mapped through an auxiliary Pre-Logit Polynomial Activation module before projecting final logits across a token field of size $V = 30,522$, managed cleanly via a localized *Hamro BPE* tokenizer framework.

3 GRADIENT SCALING

The primary challenge of deploying non-linear higher-order activations within standard backpropagation models is their inherent sensitivity to exponential gradient explosion. Because the forward pass maps operations quadratically and cubically, backpropagated error signals scale multiplicatively, introducing structural instability. To insulate the network from divergence, we implement a multi-tiered stabilization mechanism.

3.1 Global Norm-Based Gradient Clipping

To maintain macroscopic step consistency across the embedding and attention projections, the total structural gradient vector g is monitored continuously. If the global optimization metric exceeds a predefined hyperparameter threshold (1.0), the

complete gradient matrix is clipped scale-invariantly via:

$$g \leftarrow g \cdot \frac{1.0}{\|g\|_2} \quad \text{if } \|g\|_2 > 1.0 \quad (3)$$

3.2 Local Gradient Scaling and Exponential Control

While global clipping bounds macro-level anomalies, it cannot prevent micro-level polynomial branches from accelerating faster than adjacent linear projections. Thus, a localized *Constant Gradient Scaling* coefficient ($\gamma \sim 0.5$) is permanently applied to the polynomial parameters (w_d, α_k) to dampen high-frequency variations:

$$g_{\text{poly}} \leftarrow \gamma \cdot g_{\text{poly}} \quad (4)$$

Furthermore, if localized instabilities push operational values past an upper safety threshold, an automated *Exponential Gradient Control* framework activates. This protocol dynamically depresses backpropagated updates based exponentially on the magnitude of the threshold violation:

$$g_{\text{poly}} \leftarrow \begin{cases} g_{\text{poly}} \cdot e^{-\beta(\|g_{\text{poly}}\|_2 - \tau)} & \text{if } \|g_{\text{poly}}\|_2 > \tau \\ g_{\text{poly}} & \text{otherwise} \end{cases} \quad (5)$$

where τ represents the strict localized step ceiling and β is a positive tuning hyperparameter regulating the intensity of the dampening function. This dual-action stabilization strategy enables smooth, low-resource optimization profiles without sacrificing higher-order modeling expressivity.

4 EXPERIMENTAL RESULTS

We trained NanoGPT on both linear and polynomial FNN. All the experiments are available at <https://github.com/Gunjan5308A/polynomial-FNN-for-llm>

4.1 Training results on GPT 2 architecture

We trained NanoGPT on NepaliBERT by Rajan with following training config:

Table 1: Training Configuration for NepBERT-based Model

Parameter	Value
Number of Samples (n_{sample})	50,000
Number of Layers (n_{layer})	8
Number of Attention Heads (n_{head})	8
Embedding Dimension (n_{embd})	512
Context Length (block_size)	256
Vocabulary Size (vocab_size)	30,522
Batch Size	2
Gradient Accumulation Steps	32
Learning Rate	6×10^{-4}
Maximum Iterations	12,000
Learning Rate Decay Iterations	12,000
Minimum Learning Rate	6×10^{-5}
β_2	0.95
Data Type	bfloat16
Compile Mode	False

After the training model seems to be bit over fitted with the given val loss of 2.6398

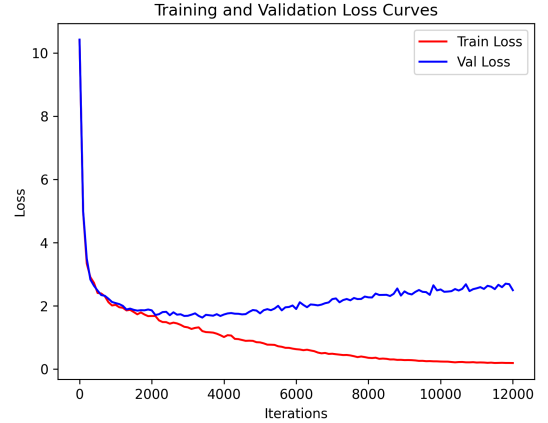


Figure 2: Loss over iteration during training.

4.2 training on polynomial FNN

With the same training config we trained a polynomial feed forward neural which gave val loss of 2.1031

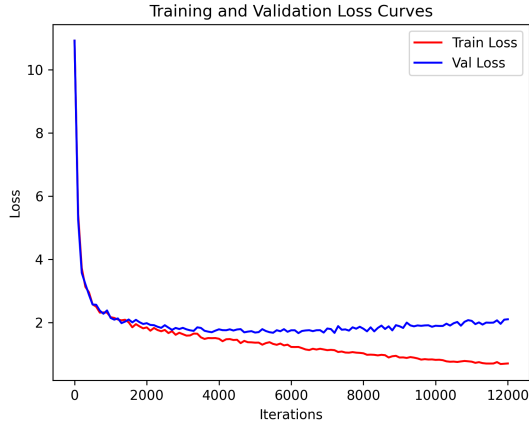


Figure 3: Loss over iteration during training.

5 conclusion

Our empirical evaluations using the NanoGPT framework on Nepali text demonstrated the distinct efficacy of the polynomial approach over standard configurations:

- **Baseline GPT-2 Architecture:** Standard linear/piecewise configurations yielded a final validation loss of **2.6398** and exhibited notable over-fitting as training progressed.
- **Proposed Polynomial FNN:** Under the exact same training configurations, our stable polynomial feed-forward network achieved a significantly lower validation loss of **2.1031**.

The lower validation floor confirms that parallelized parametric polynomials offer superior representational capacity and better generalization behavior for low-resource, morphologically rich languages like Nepali.

References

- [1] A. Vaswani, et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.
- [2] A. Karpathy, “nanoGPT,” GitHub repository, 2023. [Online]. Available: <https://github.com/karpathy/nanoGPT>
- [3] R. Pascanu, T. Mikolov, and Y. Bengio, “On the difficulty of training recurrent neural networks,” in *International Conference on Machine Learning*, 2013, pp. 1310–1318.